

Investment Questionnaire Tool

Joseph Alexius Tapang
CST2101 Business Intelligence Programming
Algonquin College
Ottawa, Canada

1. PROJECT OBJECTIVE AND PROBLEM DEFINITION

A. Business Case

Investment risk assessment has been a standard procedure in to onboard a client in financial institutions. This documents a client's response towards investment risk and the capacity to tolerate losses, his financial objectives, and investing experience. Leading investment bodies emphasizes that good risk assessment comes from investor suitability and protects both parties from misaligned recommendations[1].

Traditionally, the process of risk assessment has been done face-to-face, with paper questionnaires, or interviews guided by advisors. Given the standard process, in reality, can consume a lot of time for organizations dealing with large volume of clients. Inconsistencies, human error, documentation problems, and compilation can limit the scalability of advisory services [2]. Many financial institutions start fast tracking digital transformation wherein automated solutions for client risk assessment are becoming a norm to meet efficiency, accuracy, and compliance standards.

B. Problem Statement

Manual application processes have identified limitations and risks towards operational efficiency and integrity, therefore identifying the problem through different questions:

1. How can we improve risk customer experience and assessment processing time?
2. How can we achieve consistent risk scoring and assessment results?
3. How can we integrate the results with other banking applications for smoother processing?

C. Project Objective

This project aims to design and implement an automated solution to capture client responses and

identify investment risk assessment. The details were outlined as Business requirements below:

BR-001: Create an automated questionnaire tool that will contain investment questions and choices.

BR-002: Each choice should have a corresponding weight to be used for computation.

BR-003: Automatically determine the risk appetite (Conservative, Moderate, Aggressive) based from the calculated weights.

BR-004: Save client details, and appetite to a csv file. Reduces manual workload of advisors by improving accuracy and consistency.

It combines operational efficiency with regulatory investor suitability requirements and industry best practices, minimizing the paperwork associated with more conventional assessment methods [3].

D. Gap Analysis

To justify the reasons for initiating a project, an analysis of the current and future state was performed to identify the gaps and solutions.

Current State:

1. Financial advisor hands the paper questionnaire to the client.
2. Client shades the corresponding answer for each question in the questionnaire.
3. Client submits the completed questionnaire to financial advisor.
4. Financial advisor checks each answer weight against the answer matrix, computes for the risk score, then identifies if the client is Conservative/ Moderate/ Aggressive.
5. Financial advisor consolidates the client application documents along with the risk assessment questionnaire and submits all documents for encoding at the end of the business day.

Future State:

1. Financial advisor loads the questionnaire tool in a browser in a device e.g. tablet, and hands it over to the client.

2. Client selects each answer in the form.
3. Client clicks the submit button once done.
4. The questionnaire tool automatically determines the weight from the answer matrix, computes for the risk score, and identifies if the client is Conservative/ Moderate/ Aggressive.
5. Financial advisor can confirm and submit client's information, risk score, and risk assessment into a csv file.

Gaps:

1. Paper based questionnaire needs to be printed in paper, incurring additional cost of approximately 1 cent per page [4].
2. Manual computation of risk score can consume up to 15 minutes or equivalent to 25% of an advisor's hourly rate.
3. Manual risk score calculation has a risk of financial advisor's misinterpretation and error that can negatively impact client experience.
4. Additional effort needed to encode consolidated client information and profile into another system, incurring additional manpower costs, e.g. 1 hour salary equivalent of an office staff.
5. Incomplete application details can lead to compliance issues.

Action Plan:

1. Build a software tool that will capture and eliminate paper applications.
2. Design the tool to automatically compute and determine the client risk profile.
3. Allow all responses to be consolidated into one file that can be used by other systems in the bank.
4. Apply validations to ensure that details are complete.

2. PLANNING, DESIGN, METHODOLOGY, TOOLS

Requirements Elicitation: After the problem, gaps between current and future states, and business requirements were identified, the following functional requirements have been formulated:

FR-001: Build a python program that will contain the investment questions and choices.

FR-002: Create a matrix that contains the answers and weights.

FR-003: Calculate the risk appetite from the weights obtained from the answers. The output shall be based from the following scores:

Table 1. Risk score and risk appetite

Score Range	Risk Appetite
7 to 35	Conservative
36 to 54	Moderate
55 to 75	Aggressive

FR-004: Save the client id, date, and risk appetite into csv file.

Design: The application design was made using streamlit library and consists of two pages, one for client forms and one for admin. The client forms contain three tabs: Personal Information to input client personal details, Questionnaire tab to contain the investment questionnaire questions, and the Risk Assessment Results that will display the risk score and client profile. This is where the data can be appended into an excel file.

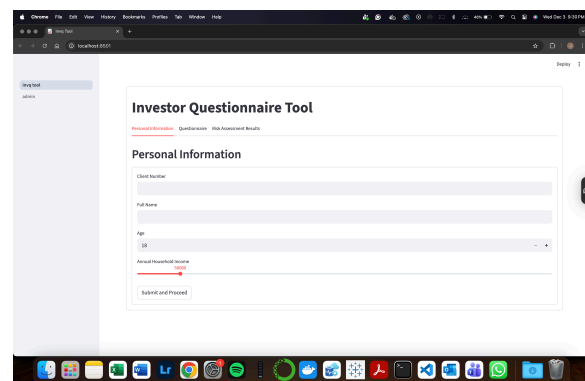


Figure 1. User Interface

Methodology: This project follows a standard software development lifecycle methodology (Waterfall) with the following phases: Planning, Analyzing, Designing, Development, and Testing. No deployment shall be made as this project is for academic purposes only.

Tools:

Questionnaire: The questionnaire shall be based from Vanguard investment questionnaire that contains the questions and weights used in this tool. This is also the source of truth used in testing the accuracy of the tool [8].

Package Manager: Anaconda Navigator was used in this project. Applications within the interface can be launched without the need of a command line interface (CLI). Additionally, packages within the Anaconda distribution and environments can be managed within the interface. Additional packages within the Anaconda website can be searched using the interface [5]. This interface is preferred and was used

throughout the lecture and laboratory phases in this course.

User Interface: Jupyter notebook was used in this project. As a web-based interactive environment for coding, flexible arrangement of workflows makes it a great tool for data science and machine learning. As an open source product that is based from IPython Project, Jupyter notebook efficiently utilizes python and its libraries through its design and structure [7].

Programming Language: Python was the programming language used for building the tool. It is an object oriented and interpreted high-level programming language that was built in data structures and combined with dynamic typing and dynamic binding [6]. Codes can be executed per cell and run to produce a granular result without the need to execute all the codes. Results from each cell can be used in other sections of the whole program that makes it ideal for step by step programming. It can also be executed as one full program when saved as a python file.

Libraries: The libraries used were streamlit to build the user interface and interactive components as well as running the program as a web application. Pandas, numpy, matplotlib, plotly, datetime, and csv libraries were also used to load the data from the csv into the dataframe and manipulate according to the desired output. Matplotlib was then used for generating visualizations.

3. CODE IMPLEMENTATION

A. Logic

Below are the sequence of events and conditions that takes place in the application.

Running the application, importing libraries, initializing the variables, and calling the main application function:

1. Application runs, loads the landing page. Refer to *Figure 1*.

2. Import all necessary libraries.

3. Define a function `change_submit_session_state()` : this function changes the session state key named `application_stage`. This key and value is used to enable other tabs once the submit button of the current tab has been clicked.

4. Initialize global variables.

5. Initialize session variables: these variables act as storage throughout the application that can be reused as long as the application has not been refreshed.

- `application_stage` – contains integers as value and used as a condition to represent the application state. Initial value is 1.
- `total_weight` – contains a float value that stores the total calculated weight from the client's choices. Initial value is 0.
- `risk_assessment` – contains a string value that stores the client's risk assessment. Initial value is ''.

6. Define a function `def main_app()`: this function serves as the main function of the application that contains the body of the program. This will only run upon execution of calling the `Main_App()` function.

7. Calling the main function: the `if __name__ == "__main__":` statement that focuses the execution of code under the main function. It helps organize the body of the application into one function and removes the risk of running unwanted codes.

Body

8. `main_container` – this code creates a container that houses all the tabs and forms. There tabs were created inside the container: `tab_personal_info`, `tab_questions`, and `tab_results`.

9. `tab_personal_info` – this tab contains the form where the client inputs their personal information, stored in the following variables:

- `clientNumber` – string. Stores client number from textbox.
- `clientName` – string. Stores client name from textbox.
- `age` – integer. Stores age from number input field.
- `income` – integer. Stores income from slider input field.
- `tab_personal_info_form_submitted` – boolean. Stores the value true when submit button is clicked. also triggers the `change_submit_session_state()` function to update the value to 2. This will allow the next tab to display the contents.

10. `tab_questions` – this tab contains the questions and answer choices that the client will select, stored in the following variables below. This tab will only be available if the `application_stage` value is already 2.

- `df_questions` – dataframe. Stores the information read from `questions.csv`. this is under a try statement and will return an exception if file is not found.
- `answer_list` – this list is initialized first, then used to store the clients answers.
- `question` – string. stores the question of the current row, under the 'Question' column from the dataframe.
- `option_2col` – list. gets the options starting from the second column. If there are null values within the selection, these shall be dropped and saved into another list named `option_list`.
- `option_list` – list. contains all non_null values to be used as choices for the question.
- `rkey` – string. contains an assigned key for the radio button group. Naming convention is `roptiongroup_{index number}`
- `selected_option` – string. stores the selected radio button from the radio button group.
- `answer_list.append(selected_option)` – this appends the currently selected option into the list named `answer_list`.
- `tab_questions_submitted` – boolean. Stores the value true when submit button is clicked. also triggers the `change_submit_session_state()` function to update the value to 3. This will allow the next tab to display the contents.

This section gets the equivalent letter of the selected answers that is a column name from the `weights.csv` file.

- `equivalent_letter` – list. Initialized in the beginning and shall be used to store the equivalent alphabetical choice of the answer.
- `dataframeqrow` – list. Stores the row of the current index number of the list `answer_list` that is under an enumerate loop.
- `columnname` – string. Contains the column name of the matching element name from the list `answer_list`. The resulting column name
- `equivalent_letter.append(columnname)` – appends the obtained column name into the list `equivalent_letter`.

This section gets the equivalent weight from each of the index number of the equivalent letter and the letter in the `weights.csv` file.

- `df_answers` – dataframe. Stores the information read from `weights.csv`. this is under a try statement and will return an exception if file is not found.

- `equivalent_weights` – list. Initialized at the beginning and shall be used to store the equivalent weights of the answer to every question.
- `weight` – float. Contains the matching weight based from the location in the dataframe using the index number representing the row(question) and the element(answer letter as column) to retrieve the cell.
- `equivalent_weights.append(weight)` – this appends the obtained weight into the list `equivalent_weights`.
- `total_weight` – stores the sum of all the obtained weights in the list `equivalent_weights`.
- `st.session_state.total_weight` – session key that stores the total weight that can be used throughout the application process.

9. `tab_results` – this tab contains the results after passing the `total_weight` into a series of if, elif statements to determine the parameters for the donut chart. Contains the following variables:

- `graphlabels` – list. Initialized at the beginning. Contains the labels parameter of the donut chart.
- `graphvalues` – list. Initialized at the beginning. Contains the values parameter of the donut chart.
- `graphcolors` – list. Initialized at the beginning. Contains the colors parameter of the donut chart.
- `riskassessment` – list. Contains the client risk assessment. Value is defined based from the satisfied if elif condition.
- `st.session_state.risk_assessment` – session state key to store the risk assessment that can be used throughout the application process.

The table will determine the corresponding risk assessment value based from the risk score range:

Table 2. If elif values

Total Weight From	Total Weight To	graphvalues	graphlabels	graphcolors	
7	22	100	-	Bonds	darkgoldenrod
23	28	80	20	Bonds,S tocks	darkgoldenrod,teal
29	35	70	30	Bonds,S tocks	darkgoldenrod,teal
36	41	60	40	Bonds,S tocks	darkgoldenrod,teal
42	48	50	50	Bonds,S tocks	darkgoldenrod,teal
49	54	40	60	Bonds,S tocks	darkgoldenrod,teal

55	61	30	70	Bonds,S tocks	darkgolden rod,teal
62	68	20	80	Bonds,S tocks	darkgolden rod,teal
68	75	-	10 0	Stocks	teal

- confirmed_risk_profile_submitted – boolean.
Sets to true once clicked. on true, captures the date today, client details, total weight, risk profile and appends to csv file named responses.csv. it is under a try catch statement that returns an error when unable to write at the file.

Visualization: The donut chart shall depend on the values determined from the if elif statements and be displayed in the tab_results section along with the risk score and risk assessment.

4. RESULTS

This section contains the testing done through test cases and the results obtained. All test cases were mapped to the functional requirements and business requirements and therefore achieve a 100% completion and acceptance rate, as satisfied during the demo session with the professor.

Table 3. Test scenario, cases and results

BR #	FR #	Test Scenario #	Test Scenario	Test Case #	Test Case	Expected Results	Result (Pass/Fail)
BR - 001	FR - 001	TS-001	Create a questionnaire that will contain investment questions and choices	TC-001	Run investment questionnaire application	Application should run	Pass
BR - 002	FR - 002	TS-002	Each choice should have a corresponding weight to be used for computation	TC-001	Answer weights automatically loads into the program	Calculation	Pass
BR - 003	FR - 003	TS-003	Automatically determine the risk appetite (Conservative, Moderate, Aggressive) based from the calculated weights.	TC-001	Client answers questionnaire with the following answers:	Risk profile is Conservative	Pass
					Q1 - A	0	
					Q2 - A	0	
					Q3 - A	0	
					Q4 - A	1	
					Q5 - A	6	

					Q6 - A	5	
					Q7 - A	5	
					Q8 - A	1	
					Q9 - A	1	
					Q10 - A	1	
					Q11 - A	1	
					Total Weight	21	Pass
				TC-002	Client answers questionnaire with the following answers:	Risk profile is Moderate	
					Q1 - E	12	
					Q2 - E	8	
					Q3 - E	7	
					Q4 - A	1	
					Q5 - A	6	
					Q6 - A	5	
					Q7 - A	5	
					Q8 - A	1	
					Q9 - A	1	
					Q10 - A	1	
					Q11 - B	2	
					Total Weight	49	Pass
				TC-003	Client answers questionnaire with the following answers:	Risk profile is Aggressive	
					Q1 - F	17	
					Q2 - E	8	
					Q3 - E	7	
					Q4 - D	6	
					Q5 - A	6	
					Q6 - A	5	
					Q7 - A	5	
					Q8 - C	5	
					Q9 - B	3	
					Q10 - A	1	
					Q11 - A	1	

					Total Weight	64	Pass
BR-004	FR-004	TS-004	Save the client id, date, and risk appetite into csv file.	TC-001	Click submit and confirm button	The client details, risk score, profile, and date today should be appended to responses. csv	Pass

5. INSIGHTS AND RECOMMENDATIONS

At the end of this project, the key insights are as follows: The business case was gathered, therefore identifying the problem that the project needs to solve. The business requirements were identified, the gaps were identified, and the action steps were decided to address the gaps, and the solution was to build an automated tool to minimize the processing time and achieve consistency. Then the functional requirements were defined to provide a granular expectation of the functionalities that will serve as a guideline for the design components. The programming environment and the tools were identified, including the libraries to build the program. The program was then documented for reference and clearer explanation of the logic for traceability. Lastly, testing was done through a demo session and the test cases were satisfied, signifying completion of the project.

It is recommended that the tool can be further enhanced and include the succeeding modules and processes in the investment journey of the client with the financial institutions and showcase the additional skills to be learned in the upcoming courses.

6. REFERENCES

- [1] CFA Institute, *Standards of Practice Handbook*, 11th ed., Charlottesville, VA: CFA Institute, 2020.
<https://www.cfainstitute.org/sites/default/files/-/media/documents/code/code-ethics-standards/standards-practice-handbook-12th-edition.pdf>
- [2] Deloitte, "The future of financial services: Digital transformation and operational modernization," Deloitte Insights, 2022.
<https://www.deloitte.com/global/en/Industries/financial-services/perspectives/the-future-of-the-financial-services.html>
- [3] Ontario Securities Commission (OSC), "Client-Focused Reforms: Enhancing investor protection," OSC Guidelines, 2023. https://www.osc.ca/sites/default/files/2023-08/csa_20230803_31-363_client-focused-reforms.pdf
- [4] Errera, Rob. (2024, April 7). *Printing Costs: How to Accurately Calculate Your Printing Cost per Page*. Toner Buzz.
<https://www.tonerbuzz.com/blog/printing-costs/>
- [5] Anaconda. (n.d.). *Anaconda Navigator*. Anaconda Website.
<https://www.anaconda.com/docs/tools/anaconda-navigator/main>
- [6] Python Software Foundation. (n.d.). *What is Python? Executive Summary*. Python Website.
<https://www.python.org/doc/essays/blurb/>
- [7] Project Jupyter. (n.d.). *About Us*. Jupyter Website.
<https://jupyter.org/about>
- [8] Vanguard questionnaire. (n.d.). Vanguard investor questionnaire.
<https://institutional.vanguard.com/content/dam/inst/iig-transformation/offers/vrpa/pdf/participant-education-resources/investor-questionnaire.pdf>